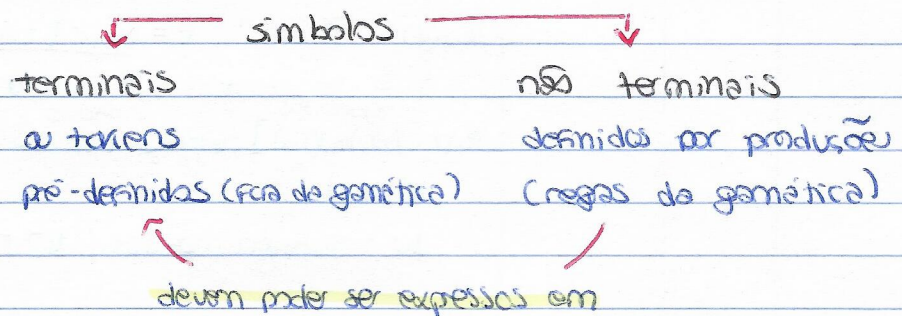


2. ANTLR4 # teórico-prático

construção de gramáticas

programação simbólica que a símbolos faz corresponder sequências de (outros) símbolos

regras
= produções



padrões

sequência de elementos

opcional zero ou uma ocorrência

repetitivo (zero ou uma) ou +

alternativa escolha entre diferentes alternativas

recursão definição recursiva de elementos

notação para regras <symbol> : <meaning>;

estrutura léxica

análise léxica conversão de seq. de caracteres em elementos léxicos

// comentários // uma linha /* multilinha */
/* JAVADOC * */

identificadores primeiro caractere letra seguido de [0-9] e -

minúsculo regra sintática
maiusculo regra léxica

literals

delimitados por películas

não há distinção entre caracteres e strings

palavras

import, fragment, lexer, parser, grammar, returns

reservadas

locals, throws, catch, finally, mode, options,
tokens, skip, rule

ações blocos de código escritos na linguagem de destino entre chavetas, sendo executados no contexto onde aparecem

exemplo

```

stat: assign | e = expr [false]
    { $at.println ($e.v); }
assign: ID '=' e = expr [true]
    { $at.println ($ID.text + "=" + $e.v); }
expr [boolean a] returns [int v]: INT {
    argumentos if ($a) retorno ...;
    $v = Integer.parseInt ($INT.text);
}

```

obs se houverem strings ou caracteres com chavetas não é necessário escape, nem se as chavetas forem balanceadas **#greedy**

regas
léxicas

regras começam por letra maiúscula!

[Fragment] RULENAME : ... ;

↳ visível apenas no lexer

expressões	X	regra X	X?	regra opcional
regulares	'...'	literal (texto)	X+	1 ou + ocorrências
	[...]	conjunto de caracteres	X*	0 ou + ocorrências
	'a'...'z'	intervalo de caracteres	~ X	tudo os caracteres menos
	X...'Z'	intervalo de regras	.	qualquer caracter
	(...)	subregra	X*?Y	X até aparecer Y
	{...}	ações		
	X1 ... Xn	alternativas		
	{pt?}	avalia predicado semântico p (se falso, regra é ignorada)		

padrões
comuns

identifiers

ID: LETTER (LETTER|DIGIT)*;

fragment LETTER: 'a'...'z' | 'A'...'Z' | '_' ;

" DIGIT: '0'...'9' ;

numbers

INT: DIGIT+;

FLOAT: DIGIT+'.'+DIGIT+ | '.'+DIGIT+;

strings

STRING: '"' (ESC | .) *? '"' ;

fragment ESC: '\\"' | '\\\\' ;

comments

LINE: '//' . *? '\n' → skip;

COMMENT: '/*' . *? '*/' → skip;

whitespace

WS: [\s\r\n]+ → skip;

análise gananciosa é feita por omissão e gera tokens com o maior tamanho possível

↓ solução

~~. *~~

. * ?

✓

exemplo "abc" "def" seriam <string "abc" "def"> com . *
e <string, "abc"> <string, "def"> com . * ?

Estrutura
sintática

revisão

análise sintática valida a sequência de elementos lexicais

as regras lexicais e misturadas podem aparecer misturadas, não interessando a ordem, com exceção dos casos de ambiguidade, em que a primeira definição tem prioridade

os tokens definidos em regras sintáticas têm precedência sobre os definidos explicitamente em regras lexicais

[parser | lexer] grammar <name>; o arquivo deve chamar-se <name>.g4
opcional, faz separação das gramáticas sintática e lexical

options { tokenVocab = <name>lexer; }

permite definir opções para analisadores (no ex. a origem dos tokens)

import ... ; importação de gramáticas

tokens { ... } permite associar identificadores a tokens

@x@>} ... { permite definir ações no pré-âmbulo da gramática
 @header antes da declaração da classe @members dentro da classe
 @parser: header ... restrições dos analisadores são possíveis

regras sintáticas

~~regras~~ começam com letra minúscula!

são traduzidas num método da linguagem de destino
 pelo que podemos associar variáveis locais, regras com nome
 alternativo e até nomear as diferentes alternativas de cada uma
 ↳ gera informação de contexto por cada uma

Exemplo `def [broken crash] returns [double average = 0] locals [int sum = 0] int count = 0
 '(' (JNT { count++ sum += $JNT.nr; }) * ')' {
 if (count > 0) {
 $average = (double) $sum / $count;
 else if ($crash) {
 //ERROR
 }
 };`

expressões regulares

n regra n	$n?$ regra opcional
$ny \dots z$ sequência de regras	n^* 0 ou +
$(\dots)$ subregra	n^+ 1 ou +
	$n y z$ alternativas

padrões + comuns

sequência $ny \dots z$
 '[' JNT+ ']'
 seq. com terminador (instruction ';') *
 seq. com separador expr (',' expr) *
 escolha instruction : conditional | loop | ... ;
 nesting expr : '(' expr ')' | JD;

precedência | é dada às regras declaradas primeiro!

associatividade

por defeito à esquerda
 expr : <assoc = right> expr '^' expr ;
 reverte por associatividade à direita

herança de gramáticas

as regras são herdadas, excepto quando definidas na gramática descendente

vip

obs

os imports são feitos por ordem, pelo que a expressão `import G1, G2;` não vai importar as regras de G2 definidas em G1, pois assume-se que já estão definidas na gramática descendente aquando da importação de G2

mais
sobre
ações

são executadas na fase de análise sintática da gramática

podemos adicionar a cada regra dois blocos, cuja execução precede ou sucede, respetivamente, o reconhecimento da regra

`rule ... @init { ... } @after { ... } ;`

gramáticas
ambíguas

nas linguagens de programação, feitas para ser interpretadas e executadas por máquinas não há espaço para ambiguidades

analisador léxico

- por omissão, escolhe o token que consome o máximo número de caracteres de entrada
- dá prioridade aos tokens definidos primeiro
- tokens definidos implicitamente na gramática sintática têm precedência sobre todos os outros

analisador sintático

- as alternativas definidas primeiro têm precedência sobre as restantes
- por omissão há associatividade à esquerda
- predicados semânticos

predicados
semânticos
`{ ... }`

utilização de informação semântica para orientar o analisador sintático através de inserção na gramática de código na linguagem de destino

permitem ativar / desativar porções das regras gramaticais durante a própria análise sintática

exemplo | sequence: JNT numbers [\$JNT.int]
 numbers [int cont] : locals [int c = 0] ;
 ({ < \$cont ? JNT { \$c++ ; } }) +
 2 3 1 3 5 6 7
 tokens numbers tokens numbers

separar
analisadores

podemos separar a definição dos analisadores léxico e sintático, potenciando assim a sua reutilização

- regras
- cada gramática indica o seu tipo no cabeçalho
 - os nomes das gramáticas devem acabar em lexer e Parser
 - todos os tokens implicitamente definidos sintaticamente têm de passar para o analisador léxico
 - a gramática léxica deve ser compilada antes da sintática
 - a gramática sintática deve incluir uma opção a indicar o analisador léxico

\$ antlr4 <= lexer.g4
 antlr4 <= parser.g4
 antlr4-maven -f <= compilationUnit
 -L <listener>
 -V <visitor>
 os listeners e os visitors
 são criados sobre o Parser!

lexer grammar <name>lexer;

...

lexer grammar <name>Parser;
 options {
 tokenVocab = <name>lexer
 }

\$ antlr4 <name>lexer.g4
 " " parser.g4

antlr4-build

antlr4-java <name>* java

antlr4-test <name> <mainrule>

ilhas
lexicais

permitem reconhecer um conjunto diferente de tokens consoante diferentes critérios (modos lexicais)

⚠ só se podem utilizar modos lexicais em gramáticas lêxicas

lexer grammar <name>lexer;

pushMode

ACTIONSTART: '{' → mode (INSIDE)

OUTSIDETOKEN: ~'}'+;

mode INSIDE;

ACTIONEND: '}' → mode (DEFAULT-MODE);

INSIDETOKEN: ~'{' +;

~['{}'] +;

INSIDEACTIONSTART: '{' →

pushMode (INSIDE);

permite nesting

ANT T

enviar tokens
para canais
diferentes

é possível retirar tokens da análise sintática, sem no entanto fazer desaparecer completamente esses tokens, podendo recuperar-se o texto que lhes está associado através dos canais léxicos

alternativa ao $\rightarrow$ skip;

canais léxicos $\rightarrow$ channel(n); n em N

obs o canal 0 é o do analisador sintático
é possível ter código para aceder aos tokens de um canal particular

reescrever a
entrada

para inserir, apagar e/ou modificar partes do código de entrada é usada a classe `org.antlr.v4.runtime.TokenStreamRewriter`

exemplo

```
public AdClass(CommentListener (- tokens) {  
    rewriter = new TokenStreamRewriter(tokens);  
}  
  
public void print() {  
    S out.println(rewriter.getText());  
}  
  
@Override public void enterTerminalDeclaration (- ctx) {  
    rewriter.insertBefore((ctx.start, "text" + ctx.Identifier().  
        .getText() + "end");  
    rewriter.insertAfter((ctx.stop, —);  
}  
  
↑ listener   ↓ main  
listener.print();
```

desacoplar
código da
gramática

faz-se com recurso à biblioteca de runtime, nomeadamente à classe `org.antlr.v4.runtime.tree.ParseTreeProperty`

esta contém um array associativo que permite associar nós da árvore com atributos

é uma alternativa aos atributos, reduzindo a dependência da gramática da linguagem escolhida

example

geometria

stat: expr;

expr: expr '*' expr # Mult;

Listener *

~~void~~ exitStat() { S.out.print(mapTxt.get(ctx.expr())) + '=' +
mapVal.get(ctx.expr()), }

~~void~~ exitMult() {

mapTxt.put(ctx, ctx.getText());

mapVal.put(mapVal.get(ctx.expr(0)) * ... (1),

{

* ParseTreeProperty<Integer> mapVal= new Parse...

" <String> mapTxt = new "